Robot Framework

Step-by-Step Guide for Beginners

Based on: github.com/lizethheredia/robot-framework-automation

---

Covers: Web | API | Database | Files

1. What is Robot Framework?

Robot Framework is an open-source keyword-driven test automation framework. Tests are written in plain English-like syntax, making them readable even for non-developers. It was created at Nokia Networks in 2005 and is now maintained by the Robot Framework Foundation.

Feature	Description
Keyword-driven	Tests use reusable actions written in plain English
Library ecosystem	100+ libraries: Browser, Requests, Database, SSH, and more
Cross-platform	Runs on Windows, Mac, and Linux
Extensible	Custom libraries written in Python
Rich reports	Generates HTML log and report after every run

2. Setup and Installation

2.1 Requirements

Python 3.8+ and pip are required. Verify:

```
python3 --version
pip3 --version
```

2.2 Create a Virtual Environment

```
python3.11 -m venv venv
source venv/bin/activate   # Mac/Linux
```

2.3 Install Libraries

```
pip install robotframework
pip install robotframework-browser
pip install robotframework-requests
pip install robotframework-databaselibrary

rfbrowser init    # downloads Chromium, Firefox, WebKit
robot --version   # verify
```

2.4 Save Dependencies

```
pip freeze > requirements.txt

# Anyone who clones the repo installs everything with:
pip install -r requirements.txt
```

3. Project Structure

Separate concerns into clear folders. This is the structure used in robot-framework-automation:

```
robot-framework-automation/
■■■ tests/              # Test suites (.robot files)
■   ■■■ web/            # UI tests
■   ■■■ api/            # API tests
■   ■■■ database/       # DB tests
■   ■■■ files/          # File system tests
■■■ resources/          # Reusable building blocks
■   ■■■ common/         # Shared utilities (browser wrapper)
■   ■■■ pages/          # Page Objects: locators + actions
■   ■■■ keywords/       # Business-level keywords
■■■ libraries/          # Custom Python libraries
■■■ variables/          # dev.yaml, qa.yaml
■■■ data/               # Test data: JSON, CSV
■■■ results/            # Reports, logs, screenshots (git-ignored)
■■■ requirements.txt
```

Rule: tests/ has Test Cases only. resources/ has Keywords. data/ has test data. Never mix.

4. Robot Framework Syntax

4.1 File Sections

```
*** Settings ***      # Imports, setup, teardown
*** Variables ***     # Reusable variables
*** Test Cases ***    # Actual tests (only in .robot files)
*** Keywords ***      # Reusable custom actions
```

4.2 File Extensions

Extension	Purpose	Contains
.robot	Test suite	Test Cases + Keywords
.resource	Reusable library	Keywords + Variables only (no Test Cases)

4.3 Variables

```
*** Variables ***
${SCALAR}      Hello           # Single value
@{LIST}        one  two  three # List
&{DICT}        key=value       # Dictionary

*** Test Cases ***
Variable Example
    Log    ${SCALAR}
    Log    ${LIST}[0]          # First item
    Log    ${DICT}[key]        # Dict key
```

4.4 Arguments

```
*** Keywords ***
Greet User
    [Arguments]    ${name}    ${greeting}=Hello
    Log    ${greeting}, ${name}!

*** Test Cases ***
Argument Test
    Greet User    Liz             # Hello, Liz!
    Greet User    Liz    Hi       # Hi, Liz!
```

4.5 FOR and IF

```
FOR    ${item}    IN    @{my_list}
    Log    ${item}
END

IF    '${role}' == 'admin'
    Log    Admin user
```

```
ELSE
    Log    Regular user
END
```

5. Web Testing with Browser Library

Browser Library is built on Microsoft Playwright. It supports Chromium, Firefox, and WebKit out of the box and is the recommended web testing library.

5.1 Page Object Pattern

Locators and low-level actions live in pages/. Business logic lives in keywords/. Tests only call high-level keywords.

```
# resources/pages/login_page.resource
*** Variables ***
${LOGIN_URL}       ${BASE_URL}/login
${USERNAME_FIELD}  id=username
${PASSWORD_FIELD}  id=password
${LOGIN_BUTTON}    css=button[type='submit']

*** Keywords ***
Open Login Page
    New Page    ${LOGIN_URL}

Enter Username
    [Arguments]    ${username}
    Fill Text    ${USERNAME_FIELD}    ${username}

Click Login Button
    Click    ${LOGIN_BUTTON}
```

5.2 Screenshot on Failure

```
Close Browser Session With Screenshot On Failure
    Run Keyword If Test Failed    Take Screenshot    filename=FAIL-{index}
    Close Browser
```

Use this keyword in every test teardown. Screenshots are saved to results/.

5.3 Headless Mode for CI

```
# Local: browser is visible (headless=False)
# CI: pass --variable HEADLESS:True to hide browser

Open Browser Session
    [Arguments]    ${browser}=${BROWSER}
    New Browser    ${browser}    headless=${HEADLESS}
    New Context    viewport={'width': 1280, 'height': 720}
```

6. API Testing with RequestsLibrary

RequestsLibrary wraps Python requests with Robot Framework keywords. Use it to test REST APIs.

6.1 Test Data from data/api_data.json

```
{
  "base_url": "https://jsonplaceholder.typicode.com",
  "existing_user": { "id": 1, "name": "Leanne Graham" },
  "new_post": { "title": "RF Post", "body": "Automated", "userId": 1 }
}
```

6.2 Keywords and Tests

```
# Create session once in Suite Setup
Create API Session
    [Arguments]    ${base_url}
    Create Session    api    ${base_url}    verify=True

*** Test Cases ***
Get User By ID Should Return 200
    ${data}=        Load API Data
    ${response}=    GET On Session    api    /users/${data}[existing_user][id]
    Should Be Equal As Integers    ${response.status_code}    200
    Should Be Equal    ${response.json()}[name]    ${data}[existing_user][name]

Create Post Should Return 201
    ${data}=        Load API Data
    ${response}=    POST On Session    api    /posts    json=${data}[new_post]
    Should Be Equal As Integers    ${response.status_code}    201
```

7. Database Testing with DatabaseLibrary

DatabaseLibrary connects to SQLite, PostgreSQL, MySQL, and others. This project uses SQLite — built into Python, no server needed.

```
# Test data from data/db_data.json:
# { "users": [{"id":1,"name":"Alice","email":"alice@ex.com","role":"admin"}, ...] }

Setup Database
    ${db_path}=   Setup Test Database
    Connect To Database    sqlite3    ${db_path}

*** Test Cases ***
Database Should Have Correct User Count
    ${data}=      Load DB Data
    ${expected}=  Evaluate    len($data['users'])
    ${count}=     Row Count    SELECT * FROM users
    Should Be Equal As Integers    ${count}    ${expected}

Query Should Return Correct User
    ${result}=    Query    SELECT name FROM users WHERE id=1
    Should Be Equal As Strings    ${result}[0][0]    Alice
```

8. File System Testing with OperatingSystem

The built-in OperatingSystem library handles files and directories. No installation needed.

```
# Test data from data/file_data.json:
# { "files": [{"filename":"hello.txt","content":"Hello from RF!"}] }

*** Test Cases ***
Create And Verify Text File
    ${data}=        Load File Data
    ${file_info}=   Set Variable    ${data}[files][0]
    Create File    results/files/${file_info}[filename]    ${file_info}[content]
    File Should Exist    results/files/${file_info}[filename]
    ${content}=    Get File    results/files/${file_info}[filename]
    Should Be Equal    ${content}    ${file_info}[content]
```

9. Running Tests

```
# All tests
robot --outputdir results tests/

# By module
robot --outputdir results tests/web/
robot --outputdir results tests/api/
robot --outputdir results tests/database/
robot --outputdir results tests/files/

# Specific file
robot --outputdir results tests/web/login_test.robot

# Change browser
robot --outputdir results --variable BROWSER:firefox tests/web/

# Headless
robot --outputdir results --variable HEADLESS:True tests/web/

# Open report
open results/report.html
```

File	Purpose
report.html	High-level pass/fail summary with statistics
log.html	Step-by-step execution log with screenshots
output.xml	Raw data for re-running or merging results

10. Best Practices

Use .resource for reusable code

Never put reusable keywords in .robot files. .robot is for Test Cases only.

Page Object Pattern

Locators in pages/, business logic in keywords/. Separation of concerns.

Descriptive keyword names

Use 'Login With Valid Credentials' not 'Do Login'. Keywords read like English.

Never hardcode credentials

Use .env for secrets. Add .env to .gitignore. Never commit passwords.

Read test data from data/

Store test data in data/ as JSON or CSV. Never hardcode data inline in tests.

Screenshot on failure

Always add 'Run Keyword If Test Failed Take Screenshot' in teardowns.

Keep tests independent

Each test sets up and cleans its own state. Never depend on test order.

Variables per environment

Use dev.yaml and qa.yaml. Switch environments with --variablefile.

11. Quick Reference

Built-in Keywords

Keyword	Example
Log	Log Hello World
Should Be Equal	Should Be Equal ${a} ${b}
Should Contain	Should Contain ${text} Robot
Set Variable	${x}= Set Variable 42
Evaluate	${n}= Evaluate len($my_list)
Get File	${c}= Get File data/file.json
Run Keyword If	Run Keyword If ${cond} My Keyword
Sleep	Sleep 2s

Library Quick Reference

Library	Install	Key Keywords
Browser	pip install robotframework-browser rfbrowser init	New Browser, New Page, Fill Text, Click, Get Text, Take Screenshot
RequestsLibrary	pip install robotframework-requests	Create Session, GET On Session, POST On Session
DatabaseLibrary	pip install robotframework-databaselibrary	Connect To Database, Query, Row Count, Disconnect
OperatingSystem	Built-in	Create File, Get File, File Should Exist, Create Directory
Collections	Built-in	Get From List, Length Should Be, Dictionary Should Contain Key

Resources

Resource	URL
Official User Guide	https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html
Browser Library	https://robotframework-browser.org/
This Repo	https://github.com/lizethheredia/robot-framework-automation
Live Report	https://lizethheredia.github.io/robot-framework-automation/report.html

12. Parallel & Cross-Browser Execution

Robot Framework runs tests sequentially by default — one test at a time. With Pabot (Parallel Robot Framework), you can run multiple test suites simultaneously, cutting execution time significantly.

12.1 Why Parallel?

Mode	Tests	Time (example)
Sequential (robot)	10 tests, 1 browser	37 seconds
Parallel (pabot)	10 tests, 1 browser, 3 processes	14 seconds
Cross-browser parallel	10 tests x 3 browsers	~15 seconds

12.2 Install Pabot

```
pip install robotframework-pabot
pip freeze > requirements.txt
```

12.3 Parallel — Same Browser

Run multiple test suites at the same time using the same browser. Pabot splits suites across N processes:

```
# Run with 3 parallel processes
pabot --processes 3 --outputdir results tests/web/

# Example output:
# [PID:14917] [0] EXECUTING Web.Alerts Test
# [PID:14916] [1] EXECUTING Web.Checkboxes Test
# [PID:14915] [2] EXECUTING Web.Dynamic Content Test
# ...
# Total testing: 37.0 seconds
# Elapsed time:  14.26 seconds   <-- almost 3x faster!
```

> Pabot automatically assigns suites to free processes as they finish.

12.4 Cross-Browser Parallel Execution

Run the full test suite simultaneously on Chromium, Firefox, and WebKit. This is done with a shell script that launches 3 Pabot instances in parallel and merges the results into a single report using rebot:

```
# run_cross_browser.sh
#!/bin/bash
mkdir -p results/chromium results/firefox results/webkit

pabot --processes 2 --variable BROWSER:chromium \
      --outputdir results/chromium tests/web/ &

pabot --processes 2 --variable BROWSER:firefox \
      --outputdir results/firefox tests/web/ &
```

```
pabot --processes 2 --variable BROWSER:webkit \
      --outputdir results/webkit tests/web/ &

wait   # wait for all 3 to finish

# Merge all results into one report
rebot --outputdir results \
      --output output.xml --log log.html --report report.html \
      results/chromium/output.xml \
      results/firefox/output.xml \
      results/webkit/output.xml

echo 'Done! Open results/report.html'
```

```
chmod +x run_cross_browser.sh
./run_cross_browser.sh
```

rebot is a built-in Robot Framework tool that merges multiple output.xml files into a unified report. No extra installation needed.

12.5 Architecture

How the layers work together:

Layer	Tool	Role
Orchestrator	Robot Framework	Reads .robot files, runs keywords
Parallelizer	Pabot	Splits suites across multiple processes
Browser engine	Browser Library (Playwright)	Controls actual browser per process
Browsers	Chromium / Firefox / WebKit	Each process gets its own browser instance
Result merger	rebot	Combines output.xml files into one report

12.6 Cross-Browser in CI (GitHub Actions)

The workflow runs all 3 browsers in parallel on every push to main:

```
- name: Run cross-browser tests in parallel
  run: |
    mkdir -p results/chromium results/firefox results/webkit

    pabot --processes 2 --variable BROWSER:chromium \
          --variable HEADLESS:True \
          --outputdir results/chromium tests/web/ &

    pabot --processes 2 --variable BROWSER:firefox \
          --variable HEADLESS:True \
          --outputdir results/firefox tests/web/ &

    pabot --processes 2 --variable BROWSER:webkit \
          --variable HEADLESS:True \
```

```
      --outputdir results/webkit tests/web/ &

wait

rebot --outputdir results \
      --output output.xml --log log.html --report report.html \
      results/chromium/output.xml \
      results/firefox/output.xml \
      results/webkit/output.xml
```

HEADLESS:True is required in CI because there is no display server on Ubuntu runners.